

ROiCORE — Flood Intelligence Platform

“ระบบรับรายงานน้ำท่วม ประเมินความเสี่ยงอัตโนมัติ และแสดงผลบนแผนที่สด”

⚠ ปัญหาที่พบ (Pain Point)

ข้อมูลน้ำท่วมกระจัดกระจาย ลำบาก และประชาชนรายงานเหตุฉุกเฉินได้ยากลำบาก

🎯 จุดประสงค์ของระบบ (Purpose)

รวมศูนย์รายงานภาคประชาชน ผสานภาพถ่ายดาวเทียม และแจ้งเตือนภัยแบบเรียลไทม์

📌 1. รายงานด่วน ≤ 3 Taps

เลือกระดับน้ำ → ใส่เบอร์ → ส่งพิกัด GPS ทันที ไม่ต้องพิมพ์ยาว

✳ 2. ดาวเทียม GISTDA 77 จว.

ดึงชั้นข้อมูลน้ำท่วมจากดาวเทียมครอบคลุมทุกจังหวัดทั่วไทย

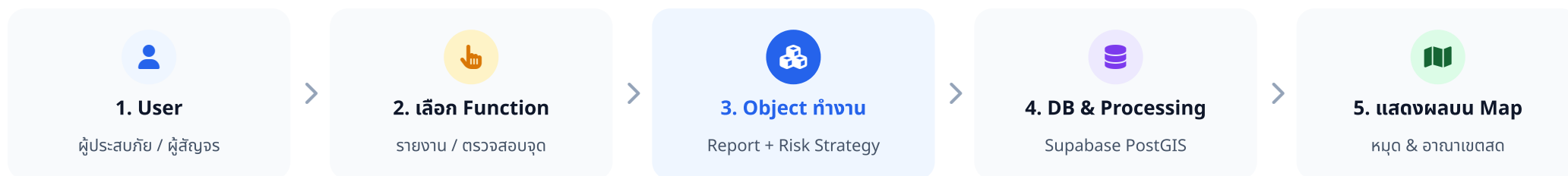
📐 3. อาณาเขตนํ้าท่วม Polygon

คำนวณและวาดขอบเขตพื้นที่วิกฤตบนแผนที่สดอัตโนมัติ

🎤 **สรุปย่อ 1 (WHAT):** "ROiCORE คือระบบรับรายงานน้ำท่วมด่วน ประเมินความเสี่ยง และแสดงผลบนแผนที่ดาวเทียมเรียลไทม์ครับ"

ระบบทำงานอย่างไร? (System Flow)

เส้นทางการเดินทางของข้อมูลแบบ End-to-End: ผู้ใช้ → ฟังก์ชัน → ประมวลผล → แสดงผล



ฝั่งผู้ประสบภัย (Flood Victim)

กดปุ่มแจ้งเหตุ → เลือก 1 ใน 4 ระดับน้ำ → ใส่เบอร์โทร → ระบบดึง GPS อัตโนมัติและส่งข้อมูลขึ้นระบบทันที

ฝั่งผู้เดินทาง (Traveler)

เปิดแผนที่ OpenStreetMap → ตรวจสอบจุดน้ำท่วมและสภาพอากาศ → กรองข้อมูลรายจังหวัดบนแดชบอร์ดโดยไม่ต้องลืกรับ

📢 **สรุปข้อ 2 (HOW):** "ระบบทำงานเริ่มจาก ผู้ใช้เลือกรายงาน → Object ประเมินความเสี่ยง → บันทึกฐานข้อมูล → แล้วแสดงผลบนแผนที่ทันทีครับ"

การออกแบบด้วยหลักการเชิงวัตถุ (OOP & OOAD)

ซ้าย: Class Diagram แสดงความสัมพันธ์ | ขวา: 4 เสาหลัก OOP กับตัวอย่างจริงในระบบ

🏗️ SIMPLIFIED CLASS DIAGRAM

```
<<abstract>> User —|> FloodVictim (ส่งรายงาน) —|> Traveler (ตรวจเส้นทาง)
FloodReport —(สร้าง)—> FloodPoint — location: GeoLocation —
riskLevel: RiskLevel FloodPoint —(ประเมิน)—> <<interface>>
IRiskStrategy ↳ CompositeRiskStrategy <<interface>> IFloodRepository
— SupabaseFloodPointRepo
```

* แยก Domain Model เป็นอิสระจาก Database ตาม Clean Architecture

🔒 1. Encapsulation (ซ่อนข้อมูลภายใน)

คลาส **GeoLocation** ซ่อนฟิลด์เป็น **private** ตรวจสอบค่าฟิลด์ให้ถูกต้องเสมอ ป้องกันการป้อนฟิลด์นอกโลก

🏠 2. Inheritance (การสืบทอดคุณสมบัติ)

FloodVictim และ **Traveler** สืบทอดจาก **User** เพื่อใช้ตัวตนเดียวกันแต่มีพฤติกรรมแตกต่างกัน

🔄 3. Polymorphism (พหุสัณฐาน)

IRiskStrategy เรียกเมธอด **calculate()** เหมือนกัน แต่คำนวณสูตรต่างกันตามกลยุทธ์ (ฝนตก/ระดับน้ำ)

🔗 4. Abstraction (การซ่อนความซับซ้อน)

IFloodRepository ซ่อนคำสั่ง SQL และเชื่อมต่อ PostGIS ไว้เบื้องหลัง Domain สิ่งบันทึกได้โดยไม่ต้องรู้ไส้ใน

🗣️ **สูตรจำข้อ 3 (DESIGN):** "เราใช้ 4 เสาหลัก OOP: Encapsulation ซ่อนฟิลด์, Inheritance แยกประเภทผู้ใช้, Polymorphism สลับสูตรความเสี่ยง และ Abstraction ซ่อน SQL ครบ"

ผลลัพธ์ระบบจริง + สรุปผลงาน (RESULT)

ระบบพร้อมใช้งานจริง สาริตถารรายงานน้ำท่วมแบบเรียลไทม์ และสรุปคุณค่าทางวิศวกรรม

🖥️ Live Demonstration (Demo จริง)

<http://localhost:8000>

- 1. เปิดแผนที่ OpenStreetMap (Leaflet.js)
- 2. กดปุ่มรายงาน → เลือก "รถเล็กผ่านไม่ได้" → กดยืนยัน (3 คลิก)
- 3. หนุดและขอบเขต Polygon สีแดงโผล่บนแผนที่สดทันที

🌟 ดีไซน์ White Modern UI: ไร้เส้นขอบ ไร้เงา กะทัดรัด สบายตา

✓ 1. ระบบทำงานได้จริงสมบูรณ์

เชื่อมต่อ Supabase PostGIS และ GISTDA 77 จังหวัดสด

✓ 2. ออกแบบด้วย OOAD เติมรูปแบบ

มี Use Case, Class Diagram, Sequence Flow ครบถ้วน

✓ 3. แบ่งความรับผิดชอบเป็น Object ชัดเจน

ปฏิบัติตาม SOLID Principles โค้ดอ่านง่ายและทดสอบได้

✓ 4. รองรับการขยายตัว (Extensible)

สามารถเพิ่มเซ็นเซอร์น้ำ หรือ API ภายนอกใหม่ได้ทันที

🎤 **สรุปข้อ 4 (RESULT):** "สรุปผลงาน ROI**CORE** ทำงานได้จริง ออกแบบด้วย OOAD แบ่ง Object ชัดเจน และพร้อมนำไปใช้งานจริงครับ ขอขอบคุณครับ"